

MEMORIA PRÁCTICA 3: CONCEPTOS AVANZADOS SOBRE BASE DE DATOS RELACIONALES

SISTEMAS INFORMÁTICOS 1

Robert Valentin Calin y Félix López Laria

Grupo 1313

ÍNDICE

- 1.Introducción y Tecnologías usadas
- 2.Rediseño y Lógica de Negocio en BBDD
- 3.Optimización de Consultas
- 4.Transacciones e Integridad
- 5.Concurrencia: Bloqueos y Deadlocks

1.Introducción y Tecnologías usadas

Esta práctica tiene como objetivo profundizar en el uso de Base de Datos y en su administración y desarrollo. En esta práctica gran parte de la responsabilidad la tiene el motor de base de datos para garantizar el correcto funcionamiento, seguridad y rendimiento como iremos viendo en cada apartado.

Para la implementación se han usado las siguientes tecnologías nuevas:

- PostgreSQL 15: Motor de base de datos relacional para la implementación de triggers, procedimientos y gestionar concurrencias

- PL/pqSQL: Utilizado para la programación de la lógica dentro de la base de datos, Para el negocio interno usado en triggers y funciones

- SQLAlchemy: Se han implementado los dos modelos de SQLAlchemy para saber Interactuar con la base de datos de las dos formas que nos ofrece

- SQLAlchemy ORM: Utilizado en el servicio de usuarios y para ello también En el módulo models para mapear clases de Python directamente a tablas de bases de datos

- SQLAlchemy Core: Utilizado en la api principal para ejecutar directamente sentencias SQL crudas, esto permite un control sobre las transacciones y ejecución de triggers

2.Rediseño y Lógica de Negocio en BBDD

Se ha tenido que hacer un rediseño ya que para la práctica se ha migrado la lógica implementada en Python hacia la base de datos para asegurar de que se cumpla independientemente de quien acceda a los datos.

Se ha creado un módulo actualiza.sql el cual hace lo siguiente.

1. En primer lugar, crearemos todas las columnas necesarias y las actualizaremos para que todo funcione correctamente. Por ejemplo, al principio de la implementación se creó la columna stock, pero se tuvo que actualizar una inicialización de 100 productos por cada stock para que las pruebas salgan correctamente.

En otro caso, se creó la columna `average_rating` el cual calcula la media de valoraciones, el problema aquí es que al crear esta columna nos olvidamos de las valoraciones anteriores antes de crear dicha columna. La solución ha sido hacer una actualización para calcular la media antigua.

2. En segundo lugar, crearemos todos los triggers pedidos

- Se ha implementado el trigger `gestionStockCarrito`. Su objetivo es garantizar la consistencia para el inventario y el carrito de los usuarios en tiempo real de tal manera que al insertar un artículo en el carrito se reduce del stock de la película y se incrementa el precio del producto en el carrito. En caso de que se borre el producto del carrito, el producto volverá al stock

Se vio la necesidad de proteger la operación de borrado mediante la función `GREATEST` para el caso de que el carrito quedara en negativo

- Se ha implementado el trigger `tr_cobrar_pedido` el cual valida que el usuario tenga el saldo suficiente antes de crear un pedido

3. Por último, se ha implementado un procedimiento, este es un método sencillo que a diferencia de un trigger no se actualiza automáticamente si no que se debe llamar desde la lógica del programa

```
1  ---CREACION DE COLUMNAS Y ACTUALIZACION DE LAS MISMAS---
2  ALTER TABLE movies ADD COLUMN stock INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0);
3  ALTER TABLE movies ADD COLUMN average_rating NUMERIC(3, 2) DEFAULT 0.00;
4  -- Añadimos un stock inicial de 100 a cada producto
5  UPDATE movies SET stock = 100;
6  -- Actualizamos la media de votos para películas antiguas ya votadas antes de este campo
7  UPDATE movies m SET average_rating = (
8      SELECT COALESCE(AVG(rating), 0)
9      FROM movie_votes v
10     WHERE v.movieid = m.movieid
11 );
12
13 -- Se añaden columnas a las tablas
14 ALTER TABLE users ADD COLUMN cart_total NUMERIC(10, 2) NOT NULL DEFAULT 0.00 CHECK (cart_total >= 0);
15 ALTER TABLE users ADD COLUMN nationality VARCHAR(100) NOT NULL DEFAULT 'España';
16 ALTER TABLE users ADD COLUMN discount NUMERIC(5, 2) NOT NULL DEFAULT 0.00 CHECK (discount >= 0 AND discount <= 100);
17 ALTER TABLE orders ADD COLUMN payment_date TIMESTAMP WITH TIME ZONE DEFAULT current_timestamp;
18
19
20 ---CREACION DE TRIGGERS---
21 -- (1) Creamos una función para el trigger pedido
22 CREATE OR REPLACE FUNCTION gestionStockCarrito()
23 RETURNS TRIGGER AS $$
24 DECLARE
25     precio_peli movies.price%type;
26 BEGIN
27     IF (TG_OP = 'INSERT') THEN
28         SELECT price INTO precio_peli FROM movies WHERE movieid = NEW.movieid;
29         UPDATE movies SET stock = stock - 1 WHERE movieid = NEW.movieid;
30         UPDATE users SET cart_total = cart_total + precio_peli WHERE userid = NEW.userid;
31         RETURN NEW;
32     ELSEIF (TG_OP = 'DELETE') THEN
33         SELECT price INTO precio_peli FROM movies WHERE movieid = OLD.movieid;
34         UPDATE movies SET stock = stock + 1 WHERE movieid = OLD.movieid;
35         UPDATE users
36             SET cart_total = GREATEST(0, cart_total - precio_peli)
37             WHERE userid = OLD.userid;
38         RETURN OLD;
39     END IF;
40     RETURN NULL;
41 END;
42
43 $$ LANGUAGE plpgsql;
44 -- (1) Creamos el trigger
45 CREATE TRIGGER actualizarCarrito
46 AFTER INSERT OR DELETE ON cart_items
47 FOR EACH ROW EXECUTE FUNCTION gestionStockCarrito();
48
```

```

53 -- (2)Creamos una funcion para el trigger pedido
54 CREATE OR REPLACE FUNCTION cobrar_pedido()
55 RETURNS TRIGGER AS $$
56 DECLARE
57     saldo_actual users.balance%type;
58     descuento_user users.discount%type;
59     precio_final NUMERIC(10, 2);
60 BEGIN
61     PERFORM pg_sleep(15);
62     SELECT balance, discount INTO saldo_actual, descuento_user FROM users WHERE userid = NEW.userid;
63
64     precio_final := NEW.total_price * (1 - (descuento_user / 100.0));
65     NEW.total_price := precio_final;
66
67     IF (saldo_actual < precio_final) THEN
68         RAISE EXCEPTION 'Pago rechazado: Saldo insuficiente';
69     END IF;
70
71     UPDATE users SET balance = balance - precio_final WHERE userid = NEW.userid;
72
73     RETURN NEW;
74 END;
75 $$ LANGUAGE plpgsql;
76 -- (2)Creamos el trigger
77 CREATE TRIGGER tr_cobrar_pedido
78 BEFORE INSERT ON orders
79 FOR EACH ROW EXECUTE FUNCTION cobrar_pedido();
80
81
82
83
84
85 -- (3)Creamos una funcion para el trigger pedido
86 CREATE OR REPLACE FUNCTION actualizar_valoracion_global()
87 RETURNS TRIGGER AS $$
88 DECLARE
89     id_pelicula_afectada movies.movieid%type;
90 BEGIN
91     IF (TG_OP = 'DELETE') THEN id_pelicula_afectada := OLD.movieid;
92     ELSE id_pelicula_afectada := NEW.movieid; END IF;
93
94     UPDATE movies
95     SET average_rating = (SELECT COALESCE(AVG(rating), 0) FROM movie_votes WHERE movieid = id_pelicula_afectada)
96     WHERE movieid = id_pelicula_afectada;
97
98     IF (TG_OP = 'DELETE') THEN RETURN OLD; END IF;
99     RETURN NEW;
100 END;
101 $$ LANGUAGE plpgsql;
102 -- (3)Creamos el trigger
103 CREATE TRIGGER valoracion automatica
104 AFTER INSERT OR UPDATE OR DELETE ON movie_votes
105 FOR EACH ROW EXECUTE FUNCTION actualizar_valoracion_global();
106
107
108
109 ---CREACION DE PROCEDIMIENTO---
110 CREATE OR REPLACE PROCEDURE calcular_media_pelicula(p_movieid INTEGER, INOUT media_final NUMERIC)
111 LANGUAGE plpgsql AS $$
112 BEGIN
113     SELECT COALESCE(AVG(rating), 0) INTO media_final FROM movie_votes WHERE movieid = p_movieid;
114 END;
115 $$;

```

3.Optimización de Consultas

Utilizando EXPLAIN podemos observar el rendimiento de una consulta

Se ha realizado un antes y un después (implementando índices para una mayor rapidez de búsqueda) usando EXPLAIN

El plan de ejecución inicial mostraba un escaneo secuencial en las tablas order y user, esto es muy ineficiente por la lentitud que presenta, además cuanto mas volumen de datos hay mas cantidad de elementos de la tabla se lee por lo cual cada vez se observaría más la ineficiencia.

La estrategia usada para solucionar este problema ha sido la indexación, es decir, se han creado dos índices para optimizar los filtros de la consulta

- I. Idx_users_nationality: Para acelerar el filtro por país
- II. Idx_orders_year: Un índice más especificativo. Aquí ante un problema durante las Pruebas se ha decidido especificar la zona horaria UTC para cumplir el requisito de PostgreSQL

Tras la creación de los índices, el coste estimado se redujo drásticamente como era de esperado.

```
1 -- Se hace un DROP para asegurar que la prueba inicial se hace desde 0 sin tener índices
2 DROP INDEX IF EXISTS idx_users_nationality;
3 DROP INDEX IF EXISTS idx_orders_year;
4 -- EXPLAIN se usa para visualizar la estrategia de optimización
5 -- Ejecutamos una consulta antes de optimizar para ver el antes
6 EXPLAIN SELECT o.orderid, o.order_date, o.total_price, u.username, u.nationality
7 FROM orders o
8 JOIN users u ON o.userid = u.userid
9 WHERE u.nationality = 'España'
10 AND EXTRACT(YEAR FROM o.order_date) = 2023;
11 -- Se crean índices para agilizar la búsqueda
12 CREATE INDEX idx_users_nationality ON users(nationality);
13 CREATE INDEX idx_orders_year ON orders ((EXTRACT(YEAR FROM order_date AT TIME ZONE 'UTC')));
14 -- Ejecutamos la misma consulta para probar la optimización una vez usados los índices
15 EXPLAIN SELECT o.orderid, o.order_date, o.total_price, u.username, u.nationality
16 FROM orders o
17 JOIN users u ON o.userid = u.userid
18 WHERE u.nationality = 'España'
19 AND EXTRACT(YEAR FROM o.order_date) = 2023;
```

Para hacer la prueba de optimización y ver los resultados de manera enfocada, deberemos de ejecutar el siguiente comando después de ejecutar el cliente.py

```
cat db/optimizacion.sql | docker exec -i postgres_si1 psql -U alumnodb -d si1
```

En nuestro programa observamos que hay un pequeño cambio al crear los índices, esto es porque no tenemos una cantidad alta de datos

```
DROP INDEX
DROP INDEX
QUERY PLAN
-----
Nested Loop (cost=0.00..2.05 rows=1 width=424)
  Join Filter: (o.userid = u.userid)
  -> Seq Scan on orders o (cost=0.00..1.01 rows=1 width=32)
      Filter: (EXTRACT(year FROM order_date) = '2023'::numeric)
  -> Seq Scan on users u (cost=0.00..1.02 rows=1 width=400)
      Filter: ((nationality)::text = 'España'::text)
(6 rows)

CREATE INDEX
CREATE INDEX
QUERY PLAN
-----
Nested Loop (cost=0.00..2.01 rows=1 width=424)
  Join Filter: (o.userid = u.userid)
  -> Seq Scan on orders o (cost=0.00..1.00 rows=1 width=32)
      Filter: (EXTRACT(year FROM order_date) = '2023'::numeric)
  -> Seq Scan on users u (cost=0.00..1.00 rows=1 width=400)
      Filter: ((nationality)::text = 'España'::text)
(6 rows)
```

4. Transacciones e Integridad

Se eliminaron las restricciones ON DELETE CASCADE del esquema de base de datos original lo que produjo a gestionar la integridad de forma natural mediante transacciones en la API

Se han creado 3 casos, uno incorrecto, uno correcto y un caso intermedio para poder verlo en profundidad

- Caso incorrecto(rollback), se ha implementado un endpoint que intenta borrar un usuario con pedidos activos sin limpiar sus dependencias. La base de datos muestra un error y gracias al bloque transaccional, el sistema ejecuta un rollback automático dejando a la base de datos en su estado original

```

549 @api_bp.delete("/borraPaisIncorrecto/<string:country>")
550 async def borraPaisIncorrecto(country):
551     async with engine.connect() as conn:
552         async with conn.begin():
553             try:
554                 q_del_users = "DELETE FROM users WHERE nationality = :country"
555                 await conn.execute(text(q_del_users), {"country": country})
556                 return jsonify({"message": "ERROR: Esto no debería ejecutarse"}), 200
557             except IntegrityError as e:
558                 return jsonify({"error": "Borrado fallido, Rollback ejecutado", "detail": str(e)}), 409
559             except Exception as e:
560                 return jsonify({"error": str(e)}), 500

```

- Caso intermedio(persistencia), se borra el carrito(commit) y posteriormente fallo el borrado del usuario(rollback). El usuario persiste, pero su carrito a desaparecido permanentemente

```

562 @api_bp.delete("/borraPaisIntermedio/<string:country>")
563 async def borraPaisIntermedio(country):
564     async with engine.connect() as conn:
565         try:
566             transaccion_a = await conn.begin()
567
568             res = await conn.execute(text("SELECT userid FROM users WHERE nationality = :c"), {"c": country})
569             users = res.fetchall()
570             user_ids = [u.userid for u in users]
571
572             if not user_ids:
573                 await transaccion_a.rollback()
574                 return jsonify({"message": "No users"}), 200
575
576             await conn.execute(text("DELETE FROM cart_items WHERE userid = ANY(:uids)", {"uids": user_ids})
577             await transaccion_a.commit()
578
579             transaccion_b = await conn.begin()
580             try:
581                 await conn.execute(text("DELETE FROM users WHERE userid = ANY(:uids)", {"uids": user_ids})
582                 await transaccion_b.commit()
583             except IntegrityError:
584                 await transaccion_b.rollback()
585                 return jsonify({"message": "Rollback parcial realizado"}), 409
586
587             except Exception as e:
588                 return jsonify({"error": str(e)}), 500
589
590         return jsonify({"message": "Fin"}), 200
591

```

- Caso correcto, se implementa un borrado manual en orden inverso, es decir, se borran los ítems después los orders después cart después votes y por ultimo el o los usuarios. Esto completa una transacción exitosamente

```

515 @api_bp.delete("/borraPais/<string:country>")
516 async def borraPais(country):
517     query_get_users = "SELECT userid FROM users WHERE nationality = :country"
518
519     async with engine.connect() as conn:
520         async with conn.begin():
521             try:
522                 result = await conn.execute(text(query_get_users), {"country": country})
523                 users = result.fetchall()
524
525                 if not users:
526                     return jsonify({"message": "No hay usuarios de ese país"}), 200
527
528                 user_ids = [u.userid for u in users]
529
530                 q_del_items = """
531                     DELETE FROM order_items
532                     WHERE orderid IN (SELECT orderid FROM orders WHERE userid = ANY(:uids))
533                 """
534                 await conn.execute(text(q_del_items), {"uids": user_ids})
535
536                 await conn.execute(text("DELETE FROM orders WHERE userid = ANY(:uids)"), {"uids": user_ids})
537
538                 await conn.execute(text("DELETE FROM cart_items WHERE userid = ANY(:uids)"), {"uids": user_ids})
539
540                 await conn.execute(text("DELETE FROM movie_votes WHERE userid = ANY(:uids)"), {"uids": user_ids})
541
542                 await conn.execute(text("DELETE FROM users WHERE userid = ANY(:uids)"), {"uids": user_ids})
543
544                 return jsonify({"message": f"Usuarios de {country} eliminados correctamente"}), 200
545
546             except Exception as e:
547                 return jsonify({"error": f"Error en transacción: {str(e)}"}), 500
548

```

5.Concurrencia: Bloqueos y Deadlocks

Se introdujo un retardo artificial con `pg_sleep(15)` en el trigger que se dedica al pago para simular un retardo y así se puedan hacer los cambios que pide la práctica en otra terminal.

Durante el proceso de pago la transacción mantiene un bloqueo de escritura sobre la fila de usuarios. Desde otra sesión se queda en espera hasta que la transacción de pago finaliza o falla. Esto garantiza el aislamiento

Se provoca intencionadamente forzando a competir a dos transacciones por quedarse con un recurso

PostgreSQL detecta el ciclo tras un periodo u aborta una de las dos transacciones lanzando e error `deadlock detected` permitiendo que la otra termine y el sistema no quede congelado

Prueba del bloqueo

1.Lanzar cliente.py

2.En otra terminal tratar de cambiar algo de el usuario como su descuento

Se comprobó que la sesión de la terminal 2 queda inmediatamente bloqueada.

Permanece en estado de espera hasta que la temrinal 1 termina su pausa y realiza el commit, entonces la terminal 2 finaliza su orden

Esto demuestra que se cumple la propiedad del aislamiento entre transacciones.

Mientras la transacción del pago esta viva ninguna otra transacción puede escribir sobre él

Para provocar el deadlock. La segunda sesión del descuento deberá operar en orden inverso, primero bloquear la tabla de pedidos y luego intentar bloquear usuarios

```
robert@robert-virtual-machine:~/Desktop/si1p3$ docker exec -it postgres_si1 psql
-U alumnodb -d si1 -c "UPDATE users SET discount=50 WHERE username='alice';"
UPDATE 1
```

Por último, un módulo generalizado para probar la práctica, a continuación, se mostrará como ha quedado el cliente actualizado

```
1 import requests
2 from http import HTTPStatus
3 import json
4 from datetime import datetime
5
6 USERS = "http://127.0.0.1:5050"
7 CATALOG = "http://127.0.0.1:5051"
8
9
10 TEST_MOVIE_ID = None
11 TEST_ACTOR_ID = None
12
13 def ok(name, cond):
14     status = "OK" if cond else "FAIL"
15     print(f"{{status}} {name}")
16     return cond
17
18 def main():
19     global TEST_MOVIE_ID, TEST_ACTOR_ID
20
21     print("# =====")
22     print("# Creación y autenticación de usuarios para el test")
23     print("# =====")
24
25     r = requests.get(f"{USERS}/user", json={"name": "admin", "password": "admin"})
26     if ok("Autenticar usuario administrador predefinido", r.status_code == HTTPStatus.OK):
27         data = r.json()
28         _token_admin = data["uid"], data["token"]
29     else:
30         print("\nPruebas incompletas: Fin del test por error critico")
31         exit(-1)
32
33     headers_admin = {"Authorization": f"Bearer {token_admin}"}.
34
35     r = requests.put(f"{USERS}/user", json={"name": "alice", "password": "secret"}, headers=headers_admin)
36     if ok("Crear usuario 'alice'", r.status_code == HTTPStatus.OK and r.json()):
37         data = r.json()
38         uid_alice, _ = data["uid"], data["username"]
39     else:
40         print("\nPruebas incompletas: Fin del test por error critico")
41         exit(-1)
42
43     r = requests.get(f"{USERS}/user", json={"name": "alice", "password": "secret"})
44     if ok("Autenticar usuario 'alice'", r.status_code == HTTPStatus.OK and r.json()["uid"] == uid_alice):
45         data = r.json()
46         _token_alice = data["uid"], data["token"]
47     else:
48         print("\nPruebas incompletas: Fin del test por error critico")
49         exit(-1)
50
51     headers_alice = {"Authorization": f"Bearer {token_alice}"}.
52
53     print("\n# =====")
54     print("# 1. CRUD de Actores y Películas (Extensivo)")
55     print("# =====")
56
57     r = requests.post(f"{CATALOG}/actors", json={"actorname": "Test Actor Temp"}, headers=headers_admin)
58     if ok("CRUD A: Crear actor 'Test Actor Temp'", r.status_code == HTTPStatus.CREATED and r.json()):
59         TEST_ACTOR_ID = r.json().get('actorid')
60
61     r = requests.get(f"{CATALOG}/actors/{TEST_ACTOR_ID}", headers=headers_alice)
62     ok("CRUD A: Consultar actor con ID {{TEST_ACTOR_ID}}", r.status_code == HTTPStatus.OK and r.json().get('actorname') == "Test Actor Temp")
```

```

73 r = requests.put(f"{CATALOG}/actors/{TEST_ACTOR_ID}", json={"actorname": "Test Actor Updated"}, headers=headers_admin)
74 ok("CRUD A: Actualizar actor con nuevo nombre", r.status_code == HTTPStatus.OK)
75
76
77
78
79 new_movie_data = {
80     "title": "Z Test Movie",
81     "description": "Película temporal para pruebas de CRUD.",
82     "year": 2025,
83     "genre": "Test",
84     "price": 1.00,
85     "stock": 10
86 }
87
88 r = requests.post(f"{CATALOG}/movies", json=new_movie_data, headers=headers_admin)
89 if ok("CRUD M: Crear película 'Z Test Movie'", r.status_code == HTTPStatus.CREATED and r.json()):
90     TEST_MOVIE_ID = r.json().get('movieid')
91
92 r = requests.put(f"{CATALOG}/movies/{TEST_MOVIE_ID}", json={"price": 5.50, "genre": "Test Updated"}, headers=headers_admin)
93 if ok("CRUD M: Actualizar precio y género", r.status_code == HTTPStatus.OK):
94     r = requests.get(f"{CATALOG}/movies/{TEST_MOVIE_ID}", headers=headers_admin)
95     ok("CRUD M: Verificar precio actualizado (5.5)", r.status_code == HTTPStatus.OK and float(r.json().get('price')) == 5.5)
96
97
98
99 print("\n *****")
100 print("# 2. Votaciones y Consultas Avanzadas (Top N)")
101 print("# *****")
102
103
104 r = requests.post(f"{CATALOG}/movies/1/vote", json={"rating": 5}, headers=headers_admin)
105 ok("Votos: Alice vota 5 a The Matrix (ID 1)", r.status_code == HTTPStatus.CREATED)
106
107
108
109 r = requests.get(f"{CATALOG}/movies/1/rating", headers=headers_admin)
110 if ok("Votos: Consultar media (ID 1)", r.status_code == HTTPStatus.OK and float(r.json().get('average_rating')) == 5.0):
111     print(f"Media The Matrix: {r.json().get('average_rating')} ({r.json().get('total_votes')} votos)")
112
113
114
115 r = requests.post(f"{CATALOG}/movies/1/vote", json={"rating": 1}, headers=headers_admin)
116 ok("Votos: Alice actualiza voto a 1", r.status_code == HTTPStatus.CREATED)
117
118
119
120 r = requests.get(f"{CATALOG}/movies/1/rating", headers=headers_admin)
121 ok("Votos: Verificar actualización de voto (media baja)", r.status_code == HTTPStatus.OK and float(r.json().get('average_rating')) < 5.0)
122
123
124
125 r = requests.get(f"{CATALOG}/movies", params={"top_n": 3}, headers=headers_admin)
126 if ok("Avanzadas: Obtener Top 3 películas más votadas", r.status_code == HTTPStatus.OK and len(r.json()) == 3):
127     top_movies = r.json()
128     ok("Avanzadas: Verificar que la película de prueba (ID {TEST_MOVIE_ID}) NO esté en el Top 3",
129         all(movie['movieid'] != TEST_MOVIE_ID for movie in top_movies))
130
131
132
133 print("\n *****")
134 print("# Distintas consultas de Alice al catálogo de películas")
135 print("# *****")
136
137
138
139 r = requests.get(f"{CATALOG}/movies", headers=headers_admin)
140 if ok("Obtener catálogo de películas completo", r.status_code == HTTPStatus.OK):
141     data = r.json()
142     if data:
143         for movie in data:
144             print(f"\t- {movie['title']} \t\t {movie['description']}")
145     else:
146         print("\tNo hay películas en el catálogo")
147
148
149
150 r = requests.get(f"{CATALOG}/movies", params={"title": "matrix"}, headers=headers_admin)
151 if ok("Buscar películas con 'matrix' en el título", r.status_code == HTTPStatus.OK and r.json()):
152     data = r.json()
153     if data:
154         for movie in data:
155             print(f"\t{movie['movieid']} {movie['title']}")
156
157
158
159 r = requests.get(f"{CATALOG}/movies", params={"title": "No debe haber pelis con este título"}, headers=headers_admin)
160 ok("Búsqueda fallida de películas por título", r.status_code == HTTPStatus.OK and not r.json())
161
162
163
164
165 movieids = []
166 r = requests.get(f"{CATALOG}/movies", params={"title": "Gladiator", "year": 2000, "genre": "action"}, headers=headers_admin)
167 if ok("Buscar películas por varios campos de movie", r.status_code == HTTPStatus.OK):
168     data = r.json()
169     if data:
170         for movie in data:
171             print(f"\t{movie['movieid']} {movie['title']}")
172             movieids.append(movie['movieid'])
173
174     r = requests.get(f"{CATALOG}/movies/{movieids[0]}", headers=headers_admin)
175     if ok("Obtener detalles de la película con ID {movieids[0]}",
176         r.status_code == HTTPStatus.OK and r.json() and r.json().get('movieid') == movieids[0]):
177         data = r.json()
178         print(f"\tdata: {data} ({data['year']})")
179         print(f"\tGénero: {data['genre']}")
180         print(f"\tDescripción: {data['description']}")
181         print(f"\tPrecio: {data['price']}")
182     else:
183         print("\tNo se encontraron películas.")
184
185
186
187 r = requests.get(f"{CATALOG}/movies/99999999", headers=headers_admin)
188 ok("Obtener detalles de la película con ID no válido", r.status_code == HTTPStatus.NOT_FOUND)
189
190
191
192 r = requests.get(f"{CATALOG}/movies", params={"actor": "Tom Hardy"}, headers=headers_admin)
193 if ok("Buscar películas en las que participa 'Tom Hardy'", r.status_code == HTTPStatus.OK and r.json()):
194     data = r.json()
195     if data:
196         for movie in data:
197             print(f"\t{movie['movieid']} {movie['title']}")
198             movieids.append(movie['movieid'])
199
200
201
202 print("\n *****")
203 print("# Gestión del carrito de Alice (CON PRUEBAS P3: STOCK Y NUEVOS ENDPOINTS)")
204 print("# *****")
205
206
207
208 stock_before = 0
209 if movieids:
210     r_stock = requests.get(f"{CATALOG}/movies/{movieids[0]}", headers=headers_admin)
211     if r_stock.status_code == HTTPStatus.OK:
212         stock_before = r_stock.json().get('stock', 0)
213

```



```

180 for movieid in movieids:
181     r = requests.put(f"{CATALOG}/cart/{movieid}", headers=headers_alice)
182     if ok(f"Añadir película con ID [{movieid}] al carrito", r.status_code == HTTPStatus.OK):
183         if movieid == movieids[0]:
184             r_stock_new = requests.get(f"{CATALOG}/movies/{movieid}", headers=headers_alice)
185             stock_after = r_stock_new.json().get('stock', 0)
186             ok(f"[P3 Trigger] Verificar stock reducido (Antes: {stock_before}, Después: {stock_after})", stock_after == stock_before - 1)
187
188     r = requests.get(f"{CATALOG}/cart", headers=headers_alice)
189     if ok(f"Obtener carrito del usuario con el nuevo contenido", r.status_code == HTTPStatus.OK and r.json()):
190         data = r.json()
191         if data:
192             for movie in data:
193                 print(f"\t{movie['movieid']} {movie['title']} - {movie['price']}")
194
195 if movieids:
196     r = requests.put(f"{CATALOG}/cart/{movieids[0]}", headers=headers_alice)
197     ok(f"Añadir película con ID [{movieids[0]}] al carrito más de una vez", r.status_code == HTTPStatus.CONFLICT)
198
199     r_stock_del = requests.get(f"{CATALOG}/movies/{movieids[-1]}", headers=headers_alice)
200     stock_before_del = r_stock_del.json().get('stock', 0)
201
202     r = requests.delete(f"{CATALOG}/cart/{movieids[-1]}", headers=headers_alice)
203     if ok(f"Eliminar película con ID [{movieids[-1]}] del carrito", r.status_code == HTTPStatus.OK):
204
205         r_stock_del_new = requests.get(f"{CATALOG}/movies/{movieids[-1]}", headers=headers_alice)
206         stock_after_del = r_stock_del_new.json().get('stock', 0)
207         ok(f"[P3 Trigger] Verificar stock respecto al borrar (Antes: {stock_before_del}, Después: {stock_after_del})", stock_after_del == stock_before_del + 1)
208
209     r = requests.get(f"{CATALOG}/cart", headers=headers_alice)
210     if ok(f"Obtener carrito del usuario sin la película [{movieids[-1]}]", r.status_code == HTTPStatus.OK):
211         data = r.json()
212         if data:
213             for movie in data:
214                 print(f"\t{movie['movieid']} {movie['title']} - {movie['price']}")
215         else:
216             print("\tEl carrito está vacío.")
217
218 r = requests.get(f"{CATALOG}/clientesSinPedidos", headers=headers_alice)
219 alice_found = False
220 if r.status_code == HTTPStatus.OK:
221     list_users = r.json().get("clientes_sin_compras", [])
222     alice_found = any(u['username'] == 'alice' for u in list_users)
223     ok(f"[P3 Endpoint] /clientesSinPedidos: Alice debe aparecer antes de comprar", alice_found)
224
225 r = requests.post(f"{CATALOG}/cart/checkout", headers=headers_alice)
226 ok(f"Checkout del carrito con saldo insuficiente", r.status_code == HTTPStatus.PAYMENT_REQUIRED)
227
228 r = requests.post(f"{CATALOG}/user/credit", json={"amount": 1200.75}, headers=headers_alice)
229 if ok(f"Aumentar el saldo de alice", r.status_code == HTTPStatus.OK and r.json()):
230     saldo = float(r.json().get('new_credit'))
231     print(f"\tsaldo actualizado a {saldo:.2f}".replace(".", ",").replace(".", "").replace("X", "-"))
232     r = requests.post(f"{CATALOG}/user/credit", json={"amount": 1000000}, headers=headers_alice)
233     if ok(f"Aumentar el saldo de alice", r.status_code == HTTPStatus.OK and r.json()):
234         saldo = float(r.json().get('new_credit'))
235         print(f"\tsaldo actualizado a {saldo:.2f}".replace(".", ",").replace(".", "").replace("X", "-"))
236
237 r = requests.post(f"{CATALOG}/cart/checkout", headers=headers_alice)
238 if ok(f"Checkout del carrito", r.status_code == HTTPStatus.OK and r.json()):
239     data = r.json()
240     ORDER_ID = data['orderid']
241     print(f"\tPedido {ORDER_ID} creado correctamente.")
242
243 r = requests.get(f"{CATALOG}/orders/{ORDER_ID}", headers=headers_alice)
244 if ok(f"Recuperar datos del pedido {ORDER_ID}", r.status_code == HTTPStatus.OK and r.json()):
245     order = r.json()
246     print(f"\tfecha: {order['date']} \n\tPrecio: {order['total_price']}")
247     print("\tContenido:")
248     for movie in order['movies']:
249         print(f"\t- {movie['movieid']} {movie['title']} {movie['price']}")
250
251 r = requests.get(f"{CATALOG}/cart", headers=headers_alice)
252 ok(f"Obtener carrito vacío después de la venta", r.status_code == HTTPStatus.OK and not r.json())
253
254 current_year = datetime.now().year
255 r = requests.get(f"{CATALOG}/estadisticaVentas/{current_year}/España", headers=headers_alice)
256 sales_found = False
257 if r.status_code == HTTPStatus.OK:
258     sales = r.json().get("sales", [])
259     sales_found = any(s['orderid'] == ORDER_ID for s in sales)
260     ok(f"[P3 Endpoint] /estadisticaVentas: Verificar que aparece la venta {ORDER_ID} en {current_year}/España", sales_found)
261
262 r = requests.get(f"{CATALOG}/clientesSinPedidos", headers=headers_alice)
263 alice_gone = True
264 if r.status_code == HTTPStatus.OK:
265     list_users = r.json().get("clientes_sin_compras", [])
266     alice_gone = not any(u['username'] == 'alice' for u in list_users)
267     ok(f"[P3 Endpoint] /clientesSinPedidos: Alice NO debe aparecer después de comprar", alice_gone)
268
269 print("\nPruebas completadas.")
270
271 print("\n# 3. Limpieza de base de datos")
272 print("\n# 4. Pruebas de Transacciones (Borrado de Pais)")
273
274 r = requests.put(f"{USERS}/user", json={"name": "pierre", "password": "secret"}, headers=headers_admin)
275 if r.status_code == HTTPStatus.OK:
276     uid_pierre = r.json().get('uid')
277     token_pierre = r.json().get('token')
278     h_pierre = f"Authorization: FBearer {token_pierre}"
279
280     requests.post(f"{CATALOG}/user/credit", json={"amount": 500}, headers=h_pierre)
281     requests.put(f"{CATALOG}/cart/1", headers=h_pierre)
282     requests.post(f"{CATALOG}/movies/1/vote", json={"rating": 5}, headers=h_pierre)
283
284 target_country = "España"
285
286 r = requests.delete(f"{CATALOG}/borrarPaisIncorrecto/{target_country}", headers=headers_admin)
287 ok(f"Transacción: Borrado Incorrecto debe fallar (400)", r.status_code == HTTPStatus.CONFLICT)
288
289 r = requests.get(f"{USERS}/user", json={"name": "pierre", "password": "secret"})
290 ok(f"Transacción: Verificar que el usuario sigue existiendo tras Rollback", r.status_code == HTTPStatus.OK)
291
292 r = requests.delete(f"{CATALOG}/borrarPaisIntermedio/{target_country}", headers=headers_admin)
293 ok(f"Transacción: Borrado Intermedio debe fallar parcialmente (400)", r.status_code == HTTPStatus.CONFLICT)
294
295 r = requests.get(f"{CATALOG}/cart", headers=h_pierre)
296 cart_empty = (r.status_code == HTTPStatus.OK and not r.json()) or (r.status_code == HTTPStatus.NOT_FOUND)
297 ok(f"Transacción: Verificar que el carrito SI se borra (Commit Intermedio)", cart_empty)
298
299 r = requests.delete(f"{CATALOG}/borrarPais/{target_country}", headers=headers_admin)
300 ok(f"Transacción: Borrado Correcto debe funcionar (200)", r.status_code == HTTPStatus.OK)
301
302 r = requests.get(f"{USERS}/user", json={"name": "pierre", "password": "secret"})
303 ok(f"Transacción: Verificar que el usuario ha sido eliminado", r.status_code == 401)
304 if __name__ == "__main__":
305     main()

```